

MediCare — Hospital Management & Appointment Scheduling System

Project Proposal
Prepared by: [Your Name] — Student, Department of Computer Science & Engineering, [University]
Course: CSE 3200 — Software Engineering / Systems Development Project
Version: 1.0
Date: August 2026

Table of Contents

- 1. Introduction
- 2. Problem Statement
- 3. Project Objectives
- 4. System Requirements
- 5. System Architecture
- 6. Implementation Details
- 7. Testing Strategy
- 8. Deployment Plan
- 9. Conclusion
- 10. References

1. Introduction

In modern healthcare, the manual management of appointments, patient records, doctor schedules, queues, prescriptions, and billing creates significant inefficiencies. Patients struggle to book consultations, doctors have no visibility of their day, and administrative staff lose hours to paperwork. Mistakes such as double-booked slots, lost records, and misplaced bills are common. MediCare is a full-stack Hospital Management and Appointment Scheduling System that digitizes the entire patient journey — from online appointment booking and queue management to electronic medical records, digital prescriptions, and payment tracking. The system is built as a MERN (MongoDB, Express.js, React, Node.js) application with a role-based interface for patients, doctors, receptionists, pharmacists, and administrators. This document proposes the complete system design, architecture, features, and implementation plan for the MediCare platform.

2. Problem Statement

Public and private clinics today face the following operational problems:

- **Double booking and slot conflicts:** Appointment books are maintained in registers or simple spreadsheets, leading to the same time slot being sold to two patients.
- **No self-service:** Patients must physically visit or call the hospital to book, reschedule, or cancel appointments.
- **Poor queue management:** No formal mechanism exists to order walk-in and appointment patients; emergency cases are not prioritized.
- **Fragmented records:** Medical history, prescriptions, and payment records are stored across separate paper files, making retrieval slow and error-prone.
- **No analytics:** Management cannot measure doctor workload, department load, revenue, or patient growth.

MediCare addresses all of these problems with a single integrated, web-based system.

3. Project Objectives

Primary Objectives

1. Provide an online booking system that guarantees **no double-booking** through database-level atomic slot claims.
2. Streamline the complete patient lifecycle: registration !' booking !' check-in !' consultation !' prescription !' payment !' records.
3. Provide role-specific dashboards for patients, doctors, receptionists, pharmacists, and administrators.
4. Give management real-time analytics and exportable reports.

Secondary Objectives

5. Support emergency queue prioritization workflow with real-time stock validation.
6. Secure patient data with JWT-based authentication, role-based authorization, and input validation.

4. Scope of the Project

In Scope

- Online patient registration and authentication
- Appointment scheduling and cancellation with automated reminders and break handling
- Prescription management with digital signatures and integration with pharmacy
- Lab test ordering, result tracking, and report generation
- Admin dashboard for user management and system statuses
- Audit logging of all critical actions

Out of Scope (Future Work)

- Online payment gateway integration (bKash, Nagad, cards)
- HL7/FHIR interoperability

5. Feasibility Study

Dimension	Assessment
Technical	All technologies (Node.js, React, MongoDB) are free, open-source, and widely documented. The team is competent in the MERN stack. Feasible.
Economic	Zero license cost. Deployment can run on shared hosting, a VPS, or a free-tier cloud. Feasible.
Operational	The workflow mirrors real hospital processes; staff roles map directly to system roles. Feasible with minimal training.
Schedule	The system is modular, allowing parallel development and incremental delivery. Feasible within a single semester.
Legal/Ethical	Patient data handling follows best-practice security; compliance with local data protection expectations is designed in. Feasible.

6. Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite	User interface
Styling	Tailwind CSS	Utility-first responsive design
Charts	Recharts	Analytics visualizations
HTTP Client	Axios	API communication with JWT refresh interceptor
Backend	Node.js + Express 4	REST API server
Database	MongoDB + Mongoose	Document storage, schema modeling, transactions
Validation	Zod	Request payload validation
Authentication	JWT (access + httpOnly refresh cookie)	Stateless, secure sessions
Testing	Jest + Supertest + mongodb-memory-server	Unit and integration tests
Security	Helmet, CORS, mongo-sanitize, rate limiting	Web security hardening

7. System Architecture

The system follows a **three-tier client-server architecture**:

React SPA
(Vite + Tailwind)

Express REST API Server

Mongoose ODM

MongoDB
(Atlas / local)

HTTPS / JSON

JWT + refresh cookie

- Client:** A single-page application served by Vite. It stores the JWT access token in memory and a refresh token in an httpOnly cookie. An Axios interceptor automatically refreshes expired tokens and retries the failed request.
- Server:** A modular Express application exposing versioned REST endpoints under `/api/v1`. Each module is organized as `routes + service + validation`.
- Database:** MongoDB with Mongoose models and indexes. Business-critical multi-document operations (e.g., slot claim) run inside transactions when connected to a replica set (e.g., MongoDB Atlas), with an automatic fallback for standalone instances.

8. User Roles and Access Control

Role	Responsibilities	Primary Access
PATIENT	Register, book/reschedule/cancel appointments, view records, prescriptions, payments	/patient/*
DOCTOR	Manage own schedule, view appointments, call queue next, run consultations, write records & prescriptions	/doctor/*
RECEPTIONIST	Check in patients, manage queues, register patients, record payments	/reception/*
PHARMACIST	Manage medicine inventory, adjust stock, dispense prescriptions	/pharmacy/*
ADMIN	Analytics, manage users/staff/ doctors/departments, reports, audit logs	/admin/*

Access control is enforced server-side through middleware ([authenticateUser](#), [authorizeRoles](#), [authorizeOwnership](#)) — the client interface alone is never trusted.

9. Module Design

The backend is organized into 15 modules, each with routes, validation, and service logic:

#	Module	Key Functions
1	Auth	Register, login, refresh, logout, forgot/reset password
2	Users	Staff/admin user management, activation
3	Patients	Patient profiles, registration, demographics, allergies
4	Doctors	Doctor profiles, department assignment, accepting-status
5	Departments	Department CRUD
6	Schedules	Weekly/one-time schedule creation, slot generation, day cancellation
7	Appointments	Book, cancel, reschedule, check-in — with atomic slot claim
8	Queues	Daily queue, call-next, status updates, emergency add
9	Medical Records	Consultation records, edit history, finalization
10	Prescriptions	Prescription creation, dispense workflow
11	Medicines	Inventory, stock in/out, low-stock & expiry filters
12	Payments	Payment recording, statuses, methods
13	Notifications	Per-user notifications, unread count, mark-all-read
14	Reviews	Patient feedback on completed visits
15	Admin	Analytics, top doctors, revenue reports, audit logs, CSV export

10. Database Design

The schema consists of 15 collections. Key relationships:


```
Department 1% % % n Doctor 1% % % n Schedule 1% % % n Slot[]
Doctor 1% % % n Schedule 1% % % n Appointment 1% % % n QueueEntry
User 1% % % 1 Patient / Doctor / Staff
Appointment 1% % % 1 MedicalRecord
Appointment 1% % % 1 Prescription (1% % % n items)
Appointment 1% % % 1 Payment
User 1% % % n Notification
```

Critical design decisions:

1. **Embedded slot array** (`schedules.slots`) is the single source of truth for availability. A slot's status moves `AVAILABLE` to `BOOKED`.
2. **Partial unique index** on `{ scheduleId, slotTime }` (excluding cancelled/no-show appointments) prevents double booking even under concurrency while still permitting rebooking of released slots.
3. **Atomic claim pattern**: Booking uses an update so only one request can claim a slot; the losing request is safely cleaned up via guarded release/rollback operations.
4. **Doctor snapshot** on appointments (`doctorSnapshot`, `operations/fee`) preserves billing and display data even if the doctor profile changes later.
5. **Queue entries** are embedded in the daily queue document with token numbers and an `orderedEntries()` accessor that sorts emergencies ahead of regular patients.

11. Key System Features

11.1 Concurrency-Safe Appointment Booking

The most technically demanding feature. When multiple patients attempt to book the same slot simultaneously, the database enforces a single winner via:

- An atomic `$setMatch` status transition (`AVAILABLE` to `BOOKED`) guarded on `appointmentId: null` so a losing or cancelled booking can never corrupt a live slot.
 - A cleanup path (composed of `cancelSlot`) guarded on `appointmentId: null` so a losing or cancelled booking can never corrupt a live slot.
- verified by a stress test that fires 8 concurrent booking requests for the same slot and asserts exactly one success.

11.2 Automated Schedule & Slot Generation

Doctors create one-time or recurring (weekly) schedules. The server generates uniform time slots between start and end times, honoring break windows, and marks past slots unavailable on the current day.

11.3 Live Queue with Emergency Priority

Reception checks patients in, which adds them to the doctor's daily queue with a sequential token number. Emergency patients jump ahead. Doctors call the next patient; status flows `WAITING` to `CALLED` to `IN_CONSULTATION` to `COMPLETED`.

11.4 Electronic Medical Records

Records capture symptoms, diagnosis, clinical notes, recommended tests, and follow-up instructions. Each edit is appended to an immutable edit history. Finalized records can no longer be modified, preserving medico-legal integrity.

11.5 Prescription-to-Pharmacy Workflow

Doctors prescribe medicines with dosage, frequency, and duration. Pharmacists dispense line-by-line; each line decrements medicine stock and validates availability, returning a conflict if stock is insufficient.

11.6 Management Analytics

Administrators see appointment volumes, status breakdowns, department loads, revenue by month/method, and patient growth, plus CSV exports for offline reporting.

12. API Design

The REST API is versioned and JSON-based with a uniform response envelope:

```
{ "success": true, "message": "...", "data": { ... } }
```

Error responses include a machine-readable `errorCode` (e.g. `SLOT_UNAVAILABLE`, `DUPLICATE_EMAIL`, `INVALID_STATUS`).

Representative endpoints:

Method	Endpoint	Roles	Purpose
POST	/api/v1/auth/login	public	Sign in, issue tokens
POST	/api/v1/appointments	PATIENT	Book appointment
GET	/api/v1/schedules/:doctorId/available?date=	public	List free slots
PATCH	/api/v1/queues/:doctorId/call-next	DOCTOR, RECEPTIONIST	Call next patient
POST	/api/v1/medical-records	DOCTOR	Create record
PATCH	/api/v1/prescriptions/:id/dispense	PHARMACIST	Dispense medicines
GET	/api/v1/admin/analytics/overview	ADMIN	KPIs dashboard
GET	/api/v1/admin/export/:type	ADMIN	CSV export

13. Security Measures

- **Authentication:** JWT access tokens (short-lived, in-memory) + rotating refresh tokens stored in httpOnly cookies; refresh tokens are hashed at rest and invalidated on password change.
- **Authorization:** Role-based middleware on every protected route; ownership checks on personal resources.
- **Input validation:** Zod schemas reject malformed payloads with 422 responses.
- **Sanitization:** `sanitize` neutralizes query injection via parameters.
- **Headers:** Helmet sets secure HTTP headers; CORS restricted to the configured client origin.
- **Rate limiting:** `RateLimit` limits for auth endpoints (20/15 min) and general API (200/15 min).
- **Secrets:** All configuration via environment variables; a `.env.example` documents required values; secrets never

- **Admitted:** Every critical action writes to the audit log with actor, role, target, and metadata.

14. Testing Strategy

The server is covered by an integration test suite (Jest + Supertest) that runs against an in-memory MongoDB server:

- **Auth & RBAC:** login success/failure, disabled accounts, role-denied requests.
- **Slot integrity:** concurrent double-booking (8 parallel requests != exactly 1 success), cancel-and-rebook, reschedule with slot release.
- **Queue logic:** token numbering, call-next ordering, emergency priority.
- **Stock rules:** prescription dispensing rejects when stock is insufficient (non-negative stock invariant).

All tests pass (`npx jest --runInBand`). The client is verified with a production build (`npm run build`).

15. Non-Functional Requirements

Requirement	Design
Performance	Indexed queries; pagination on all list endpoints (default 20/page, max 100); response times in the low hundreds of milliseconds for typical operations.
Reliability	Transactions for multi-document writes; atomic slot claims; graceful fallback to standalone Mongo.
Scalability	Stateless API server (JWT auth) != horizontal scaling behind a load balancer; MongoDB Atlas supports scaling.
Usability	Role-specific navigation, responsive Tailwind layout, empty/loading/error states throughout.

Maintainability

	Modular backend (routes/service/validation), centralized error handling, consistent response envelope.
Availability	Stateless design allows multiple instances; MongoDB replica set for high availability.

16. Deployment Plan

- 1. Database:** MongoDB Atlas cluster (replica set) or local mongod; run `npm run seed` to populate demo data.
- 2. Backend:** Node.js server exposing the API; set `NODE_ENV=production`, `MONGODB_URI`, `JWT_SECRET`, `CLIENT_URL`.
- 3. Frontend:** `npm run build` produces static assets served by Nginx (or via Vercel/Netlify); all configuration called directly.
- 4. Configuration:** All environment variables documented in `server/.env.example`.
- 5. Demo accounts** are seeded for every role for immediate evaluation.

17. Future Enhancements

- Online payment gateway integration (bKash, Nagad, card processing).
- Inpatient ward, bed, and operation theater management.
- Laboratory information system with report delivery to patients.
- Prescription and referral gateways.
- Machine learning-based appointment no-show prediction.
- Interoperability via HL7 FHIR.

18. Conclusion

MediCare delivers a complete, production-ready hospital management and appointment scheduling system built on the MERN stack. It addresses real operational pain points — double bookings, manual queues, fragmented records, and absent analytics — through careful database design, concurrency-safe business logic, role-based access control, and a polished role-specific user interface. The system is fully implemented, tested (18 passing integration tests), and seeded with realistic demo data, and it provides a solid foundation for future healthcare digitization.

19. References

1. MongoDB Documentation — Transactions, Compound Indexes, `$elemMatch`. <https://www.mongodb.com/docs>
2. Mongoose Documentation — Middleware, Indexes, Population. <https://mongoosejs.com/>
3. Express.js Documentation. <https://expressjs.com/en/4.x/api.html>
4. Tailwind CSS Documentation. <https://tailwindcss.com/>
5. RFC 7519 (2015) — JSON Web Token Standard. <https://datatracker.ietf.org/doc/html/rfc7519>
6. Zod Documentation — Schema Validation. <https://zod.dev>
7. Jest & Supertest Documentation — Testing. <https://jestjs.io>

9. OWASP Top Ten Web Application Security Risks <https://owasp.org/www-project-top-ten>
10. End of Proposal: Medicare Hospital Management & Appointment Scheduling System

